RISC-V INTERNATIONAL · RFP v1.1

Automatic Test Generation from the Sail Golden Model

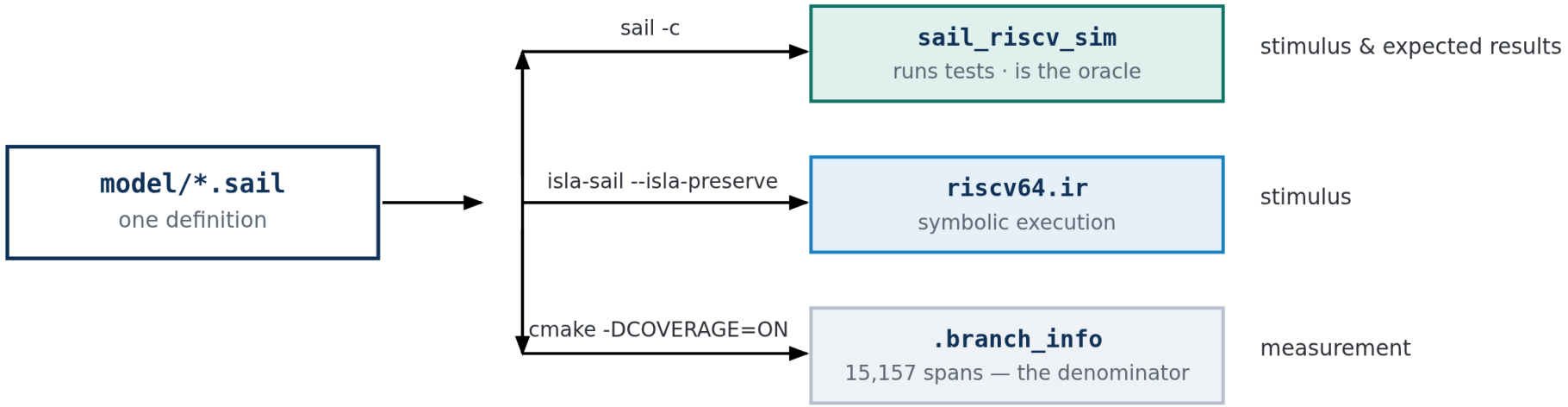
A hybrid framework: symbolic path-solving where the model can be reasoned about, concrete model-as-oracle where it cannot, and one control layer that decides which.

Technical proposal
10xEngineers

15 weeks · 2 engineers · USD 32,000
Apache-2.0

What Sail is, and what it produces

An executable definition of the architecture — simultaneously the specification, the simulator, and the source of the tests.



The same source serves generation and measurement. That is what makes coverage figures meaningful here: the thing being measured and the thing generating the tests are the same definition, not two independent interpretations of a document.

Every link back to the model — encodings, expected results, coverage denominator — moves when the architecture moves. No table in this framework needs hand-editing when an instruction is added.

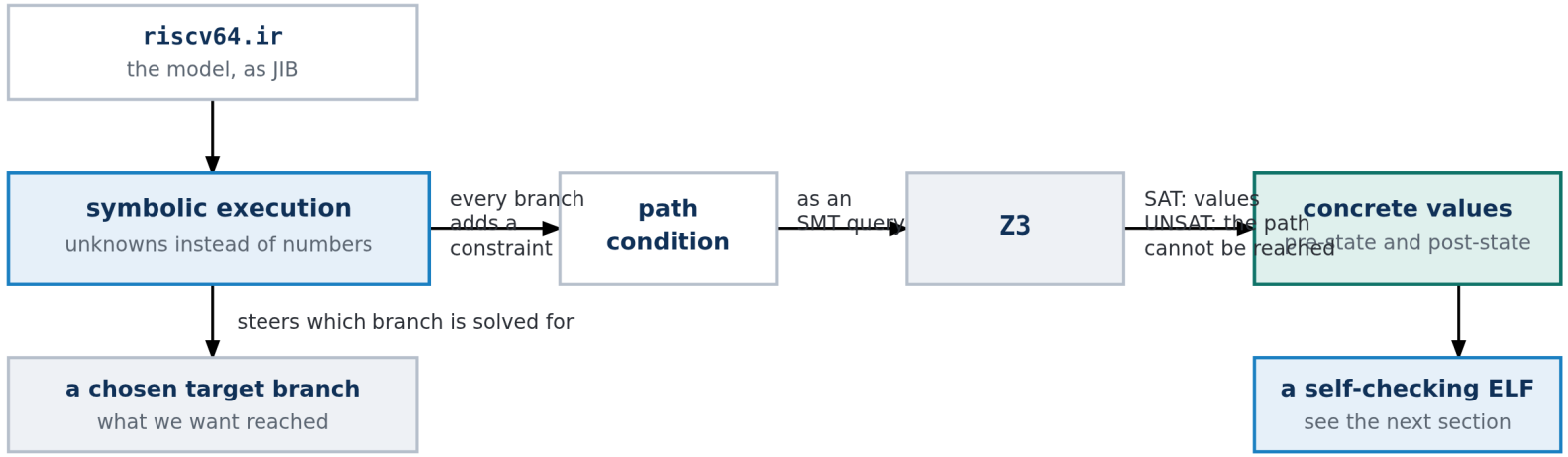
How Sail compiles, and why it matters to us

Step	What happens	Why it matters
Parse and type-check	Types depend on values — a bitvector's width is part of its type	Illegal-width code cannot be written; VLEN must be fixed at compile time
Lower to JIB	An imperative IR: no pattern matching, no polymorphism, explicit control flow	JIB is what both back ends consume
C back end	JIB → C → sail_riscv_sim	The executable Golden Model. With -DCOVERAGE=ON it records which spans ran
isla-sail back end	JIB serialised to a .ir text file rather than compiled to C	Lets isla interpret the model with symbolic values

Two compile-time facts shape the whole design. isla-sail is a Sail plugin — it links libsail and registers as a target, so it tracks the same front end as the C build. And --isla-preserve is mandatory on the entry points: without it they are pruned as dead code and the IR decodes nothing, with no error message. Configuration is baked in here too — XLEN and VLEN are frozen in the IR, not chosen at run time.

isla-testgen — solving for the inputs that reach a branch

Run the model with unknowns in place of register and immediate values. Every branch adds a condition. Hand the result to an SMT solver.



UNSAT is a result, not a failure: it is a **proof** that a branch cannot be reached, which is what lets an exclusion be justified rather than assumed

UNSAT is a result, not a failure. It proves a branch cannot be reached — which is what turns an untested span into a documented exclusion rather than a permanent, unexplained gap in the coverage figure.

From IR to assembly to ELF

Solving gives numbers. Turning numbers into a runnable, self-checking program is a separate job.

	What is produced	Detail
1	Concrete pre- and post-state	extract_state.rs asks the solver for a model and reads out every register and memory region. Nothing symbolic survives
2	A .s assembly file	The preamble that builds machine state, the instruction as raw bytes, expected-value tables, a trap handler, pass/fail reporting
3	A .ld linker script	Every section pinned at a fixed, non-overlapping address, with ENTRY(preamble)
4	An .elf	The real GNU assembler and linker are invoked — no bespoke ELF writer

Why raw bytes, not mnemonics

The test must exercise the exact encoding the model decoded — and it lets the generator emit encodings no assembler accepts, which is how illegal-instruction behaviour gets tested at all.

Why the tests check themselves

Expected values are compiled in as data; the verdict goes out through an HTIF tohost write. So one artefact runs on Sail, Spike, QEMU and RTL with no harness and no wrapper.

What isla-testgen is good at — and what it cannot do

Both lists come from building on it. The limitations are why this proposal is for a hybrid, not for scaling one engine up.

STRENGTHS

Reaches branches on purpose — name one, get a test, or a proof that nothing can

Proves unreachability — UNSAT turns a permanent gap into a justified exclusion

Knows the exact final state — the post-state comes from the same solve

Rare inputs are natural — divide-by-zero, misalignment, boundaries are just constraints

Tracks the model automatically — instructions and encodings read from the .sail sources

LIMITATIONS

Floating point — SoftFloat is external C; symbolically the operations are holes

Vector at realistic VLEN — isla's widest bitvector is 129 bits; VLEN 256 and 512 cannot be represented

Machine state — the solver derives operands, not "put a PMP entry here"

Control transfers into solved code — unsatisfiable, so privilege changes go in the preamble

Cost per test — every test is an SMT solve; wrong tool for bulk volume

isla-testgen is a precision instrument. It is right when you know which branch you want and need to be sure you reached it,
and wrong when you need many tests, wide configurations, or any part of the model it cannot represent.

The REMS lineage — use the model as an oracle

The same research programme that produced Sail and isla established the other approach, and it is much older and simpler.

1. Pick an instruction and fill its operand fields
2. Build a small program that sets up registers, executes it, and stops
3. Run it on the model — whatever state comes out is, by definition, correct
4. Emit the same program with that state written in as assertions
5. Run it anywhere. Any implementation that disagrees with the model fails

The REMS OCaml random generator exists but is not a usable base

Last commit 2019, pinned to a Sail AST shape seven years stale. We took the method and rebuilt it against the current model — which is why our instruction list comes from parsing the model.

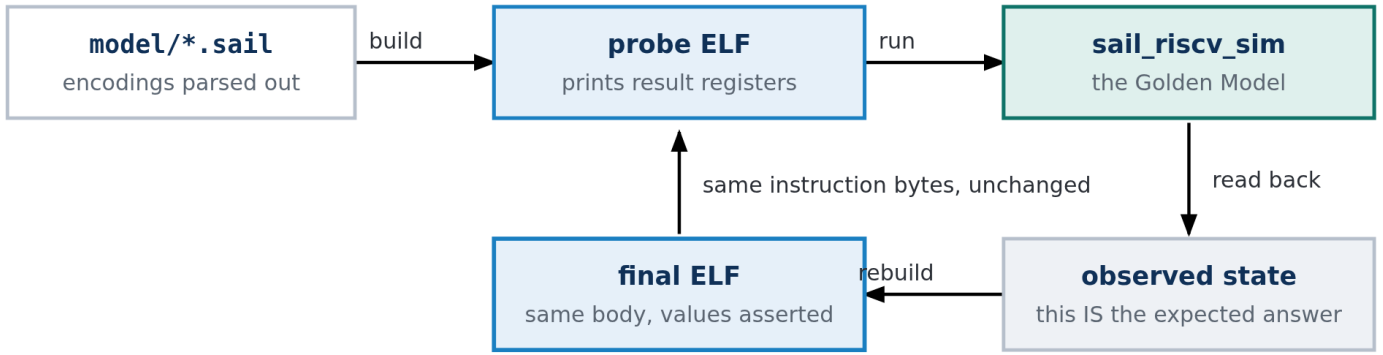
	Symbolic	Model-as-oracle
Question	What inputs reach this branch?	What does the model do with these inputs?
Needs the model to be	symbolically interpretable	runnable
Cost per test	An SMT solve	One simulator run
Reaches deep conditions	By construction	By luck
Proves unreachability	Yes	No
FP and wide vectors	No	Yes — it is just execution
Many configurations	Poorly	Well

Every row is a mirror image.

Each method's weakness is the other's strength, with no overlap in the failure modes. That is unusually clean, and it is the whole argument for combining them rather than picking one.

Our concrete oracle — the REMS method, rebuilt

Two deliberate departures: enumerated rather than random, and two ELFs sharing one body.



The instruction body is byte-identical between the two passes — only the expected-value tables differ

Not random — enumerated

The instruction list and encodings are parsed from the .sail sources, then swept systematically. Random sampling cannot tell you what it missed; enumeration gives a denominator.

Two ELFs, one body

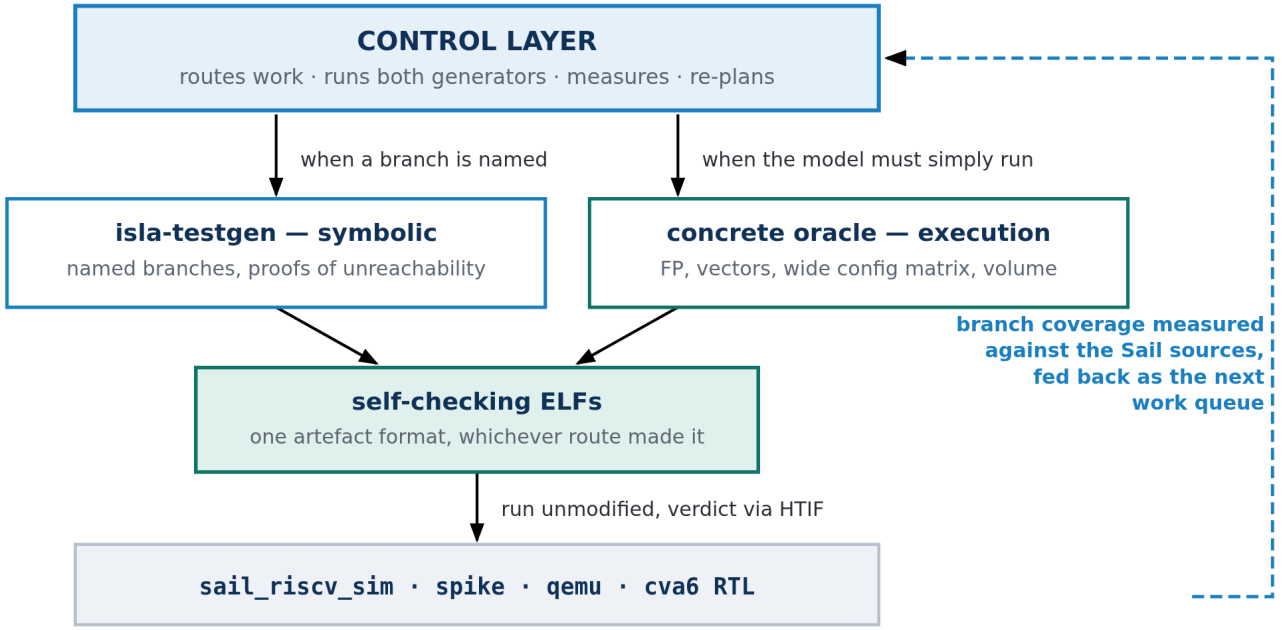
A probe ELF runs first and prints the result registers; the final ELF is the same program with those values compiled in as assertions. The expected state is observed, never predicted.

The circularity we state openly: expected values come from sail_riscv_sim, so re-running an oracle test on Sail proves only self-consistency.

The independent evidence comes from Spike, QEMU or RTL — and every figure says which simulator produced it.

The proposed framework

Two generators that fail in opposite directions, and a control layer that decides which one runs, measures what came back, and turns the shortfall into the next batch of work.



The dashed return path is what makes this a framework rather than two tools: coverage is not only reported, it is the input that

decides what gets generated next.

How the control layer operates

Four jobs — the component that turns "we have two generators" into "we have a method".

1 CONFIGURE

One Golden Model configuration, parsed once, deriving everything downstream: the IR to load, the assembler's -march, the simulator's ISA string, which extensions exist. Previously re-derived in four places, which is how they drifted apart.

2 ROUTE

Sends each target to the generator that can reach it. FP arithmetic and wide vectors to the oracle because isla cannot represent them; named privileged branches to isla because random execution reaches them only by luck.

3 EXECUTE & JUDGE

Runs each ELF on the Golden Model and on at least one independent simulator, then classifies every failure as model fault, framework fault, or correctly routed. A bare pass/fail count is not an acceptable output.

4 MEASURE & RE-PLAN

Replays the corpus against the instrumented model, reconciles every span to hit / missed / excluded-with-reason, and turns the remainder into a stimulus queue that names what is missing rather than only where.

One rule above all others: architectural state is built in the preamble, never solved for. The solver derives instruction operands; PMP entries,

page tables and privilege transitions are constructed arithmetically. Attempting the reverse fails outright.

Methodology — the scripts, and what each one does

Seventeen scripts in python-isla/, in four groups. Each is a single job with a single output, so any one can be run alone when debugging.

FOUNDATION

- paths.py — where everything lives, resolved once
- model_config.py — one config, parsed once, deriving everything
- model_opcodes.py — encodings straight from the .sail sources
- test_config.py — what each test requires, where a runner can read it

GENERATION

- opcode_sweep.py — per-Sail-file sweep, one instruction at a time
- sweep_all.py — the batch driver across extensions and both XLENs
- sweep_status.py — how far a running sweep has got
- csr_sweep.py — Zicsr across real named CSRs
- scenario_tests.py — the privileged scenarios, each with negative controls

MEASUREMENT

- coverage_report.py — replay the corpus, report Sail branch coverage
- derive_span_exclusions.py — spans no test can reach, with reasons
- coverage_analysis.py — uncovered spans → what stimulus is missing
- coverage_matrix.py — per-instruction status across both generators
- testplan.py / testplan_html.py — a plan derived from the branch structure

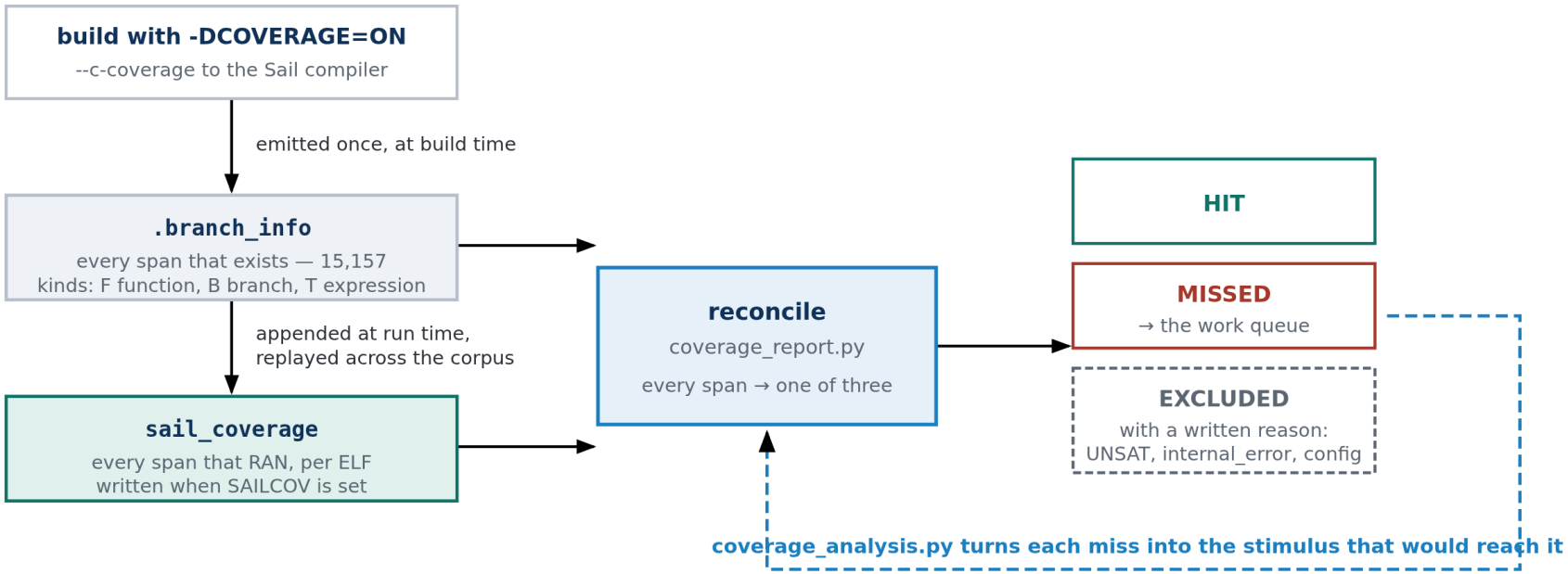
REPORTING

- regression.py — what got worse since the baseline; exits non-zero
- status_report.py — per extension, what failed and whose fault it is

scenario_tests.py is where privileged coverage lives: tests that set up architectural state rather than exercising one instruction in isolation.

The coverage flow

The RFP asks for coverage in terms of the Sail code — branches in the specification's own source, not instruction counts.

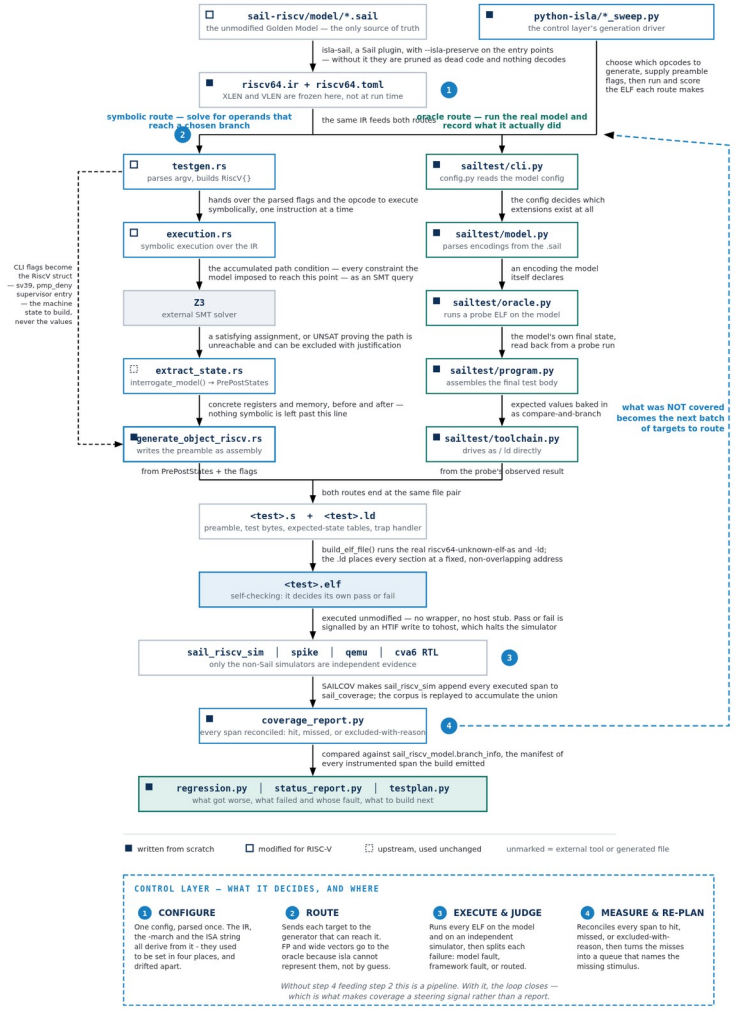


Two files do all the work: a manifest of every span the build contains (15,157), and a trace of the spans a run executed. Coverage is the ratio. The third bucket — excluded, with a written reason — is what makes a target of 100% honest rather than impossible.

A percentage is only as good as the run behind it. Span totals are stable; percentages depend on which corpus was replayed — so every figure names its corpus and its date.

The complete flow, file by file

Each arrow says what that step does. Numbered circles are the control layer's decision points; the long dashed arrow is the loop that closes.



Running the tests, and riscv-arch-test compliance

RUNNING — nothing special is required

```
sail_riscv_sim --config <cfg.json>  test.elf
spike --isa=rv64imac_zicsr        test.elf
```

A test is a plain ELF that decides its own verdict.

Reporting is an HTIF write to tohost — a nonzero odd value halts the simulator, pass and fail encoded exactly as riscv-arch-test's own macros.

An ecall was tried first and is wrong: that is the syscall-proxy convention, only active under spike pk. On bare metal it traps back into our own handler.

The ISA string, the -march and the model config all derive from one parsed configuration — a mismatch here is the most common cause of a spurious "Sail and Spike disagree".

ACT4 COMPLIANCE — six steps, all implemented

1. Cross-check our isla-sail compile config against ACT4's own sail.json
2. Adopt ACT4's exact tohost/fromhost declaration — a real, sized, backed section
3. Optional begin_signature / end_signature region behind a flag
4. START_TEST_CONFIG / END_TEST_CONFIG header for test discovery
5. Prove it through config/sail/ first — one ELF through ACT4's own build
6. Directory placement and make integration

QEMU's HTIF reads the symbol's ELF st_size and refuses to start if it is 0 — which is what a bare label gets. That is why step 2 matters and a linker constant will not do.

PMP and PMA — the case space is derived, not chosen

PMP register state is 2^4608 configurations. pmpCheck distinguishes about 44, because it composes three stages that read disjoint state.

Stage	Reads	Cases
Address match	the address and pmpaddr	17
Permission check	the access kind and R/W/X	18
Privilege override	privilege and the lock bit	4
Structural	entry 0, priority, fall-through, zero entries	5
Total	counts ADD because the stages do not interact	44, not 1,224

FIVE ADDRESS POSITIONS

1	`entirely below`	`NoMatch`
2	`straddling the low edge`	`FAULT`
3	`wholly inside`	`Match`
4	`straddling the high edge`	`FAULT`
5	`entirely above`	`NoMatch`

pmpRangeMatch compares only addr and addr+width against the two edges. Positions 2 and 4 fault regardless of permissions — the two a hand-written suite omits.

PMA works differently

Attributes are per-region fields in the model's own config JSON, not CSRs. A PMA test is "run this access under a config whose region forbids it" — no new generator capability at all.

One case no branch can see

NAPOT computes its bounds with mask arithmetic. A bug there gives wrong bounds without changing any branch — so coverage-guided where branches exist, value-guided where the code is arithmetic.

Measured 27 August 2026: one generated lw against a denied region reaches 35.8% of pmp/. Five tests reach 37.6% and 7 of 19 access arms —

but pmp_regs.sail, two thirds of the file, does not move by a single span. Register semantics need a different campaign, and that is a stated line item.

Traps, interrupts, translation and virtual memory

TRAP HANDLERS

· Emitted as assembly per test, not linked from a library

· Default: any trap fails the test — mtvec armed by the preamble's first instruction

· --expect-trap-cause: compare mcause, continue only on an exact match

· --trap-is-pass: report success from the handler — required for any violation test

· Advance mepc by 2 or 4 depending on whether the instruction was compressed

· Delegated traps read scause, not mcause — reading the wrong one is how a delegation test passes without delegation

INTERRUPTS

· Three conditions must be arranged, and getting one wrong looks like a pass

· --pending-interrupt: pending + mie, MIE left clear — tests WFI, not delivery

· --deliver-interrupt: adds mstatus.MIE, so the trap is taken

· --delegate-interrupt: a supervisor interrupt with mideleg set and stvec installed

· Machine-level interrupts cannot be delegated — mideleg hardwires MEI/MTI/MSI to zero

· An interrupt cause is the code with the top bit set — tested as a sign bit, so it stays XLEN-independent

TRANSLATION & VM

· satp.MODE decides the scheme — M-mode never translates, so tests must drop to Supervisor

· The page table is emitted into .data: 512 entries, leaf PTEs at the root, one 1 GiB gigapage each

· Two of 512 mapped — leaving 509 invalid is what makes an unmapped-page test possible

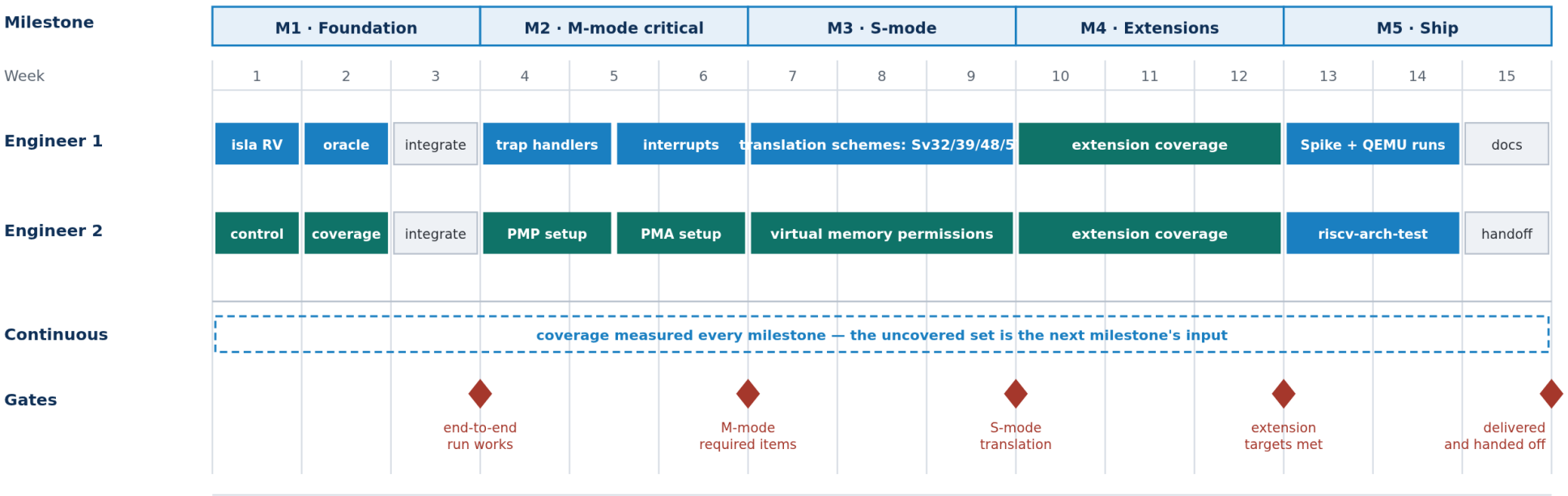
· Permissions are three gates: privilege (SUM allows loads and stores but never a fetch), shadow stack, then R/W/X after MXR

· MXR, SUM and MPRV each change the answer without touching a PTE

Virtual memory cannot be tested apart from PMP. Dropping to Supervisor arms PMP's default-deny, and the walker's own PTE reads are PMP-checked.

The joint case: deny the region the page table itself lives in — a correct model reports an access fault, a wrong one a page fault.

Fifteen weeks, two engineers, five milestones



Each milestone ends with a coverage report and a regression check. A milestone is not complete because its weeks elapsed; it is complete when its reachable spans are hit or excluded with a written reason.

Engineer 1 works on the generators, Engineer 2 on the control and measurement layer. In M1 that split lets both start on day one; from M2 the pairing

is deliberate — traps and interrupts belong together, PMP and PMA belong together, translation and its permissions are two halves of one feature.

A milestone is not complete because its weeks elapsed. It is complete when its reachable spans are hit, or excluded with a written reason.

Milestones, owners and exit gates

Milestone	Wk	Engineer 1	Engineer 2	Exit gate
M1 Foundation	1–3	RISC-V support in isla-testgen (1w) Custom oracle test generation (1w)	Control layer development (1w) Coverage collection and analysis (1w)	One instruction generated by both routes, run on Sail and Spike, with a coverage number attached
M2 M-mode critical	4–6	Trap handler generation Interrupt and exception handling	PMP setup PMA setup	Every item the RFP marks required for M-mode has generated tests with negative controls
M3 S-mode	7–9	Translation schemes — Sv32/39/48/57	Virtual memory permissions, incl. the joint PMP case	Translation active under all supported schemes; permission failures reachable, not only successes
M4 Extensions	10–12	Every extension the Golden Model supports,	except floating point and vector	Per-extension coverage reported against reachable targets
M5 Ship	13–15	Execution on Spike and QEMU Documentation finalisation	riscv-arch-test integration Handoff	The same artefacts run on three simulators; docs complete; repository handed over

Cost, and what "complete" means

COST

Duration	15 weeks — five milestones of three weeks
Team	2 engineers, in parallel throughout
Rate	USD 8,000 per month for the team
Total	**USD 32,000**
Licence	Apache-2.0 — the RFP's preferred licence

One note on the arithmetic

15 weeks is about 3.5 months; at USD 8,000/month that is ~USD 27,700. The 32,000 figure bills four months, covering the delivery weeks plus onboarding and hand-off. Strict pro-rata is 27,700 at unchanged scope — we would rather state the basis than quietly round.

DEFINITION OF COMPLETE

Coverage of the REACHABLE targets in the Sail model's branch structure.

100% is the ideal for every extension and operating mode in scope.

Three kinds of exclusion, each with a written reason:

Unreachable by construction

internal_error arms — reaching one means the model is broken

Proved unreachable

a path the solver returns UNSAT for — the strongest form, unique to the symbolic route

Not in this configuration

a span needing an extension the build does not enable; counted where it is reachable

Two figures are always published, never one. Excluding FP and vector RAISES the percentage — they are the better-covered half — so the in-scope figure alone flatters the work and the combined figure alone hides which half is the deliverable.

What already exists

This is not a plan for work that has not started. A working implementation exists today — the strongest evidence that the plan is deliverable.

- Both generators, producing self-checking ELFs from the unmodified model
- 1,280 instructions across 23 extensions, parsed from the model
- RV32 and RV64, both routes, verified end to end
- Branch coverage against the Sail sources, with justified exclusions
- Privileged scenarios — privilege transitions, PMP violations, Sv39 walks, interrupt delivery — each with negative controls
- Execution on Sail, Spike, QEMU and CVA6 RTL
- riscv-arch-test compatibility, proven through ACT4's own pipeline
- A real Golden Model defect found, fixed and filed as a PR

What the 15 weeks buys is not a prototype — it is the difference between a framework that demonstrably works and one that is complete, measured, documented, and maintainable by the community after we leave.

The gaps are known and named: PMA has no tests yet, 1 of 18 PMP access kinds is exercised, no virtual-memory permission failure is currently reachable, and the configuration matrix runs two of sixteen. Each is a line item above, not a discovery waiting to happen.